Reviewer Pack — Minimal Rank of Noncrossing Partitions vs Read-Twice BP Size…

Entry fd64effc3432 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 14:37:28 UTC

Minimal Rank of Noncrossing Partitions vs Read-Twice BP Size with Algebraic Geometry Invariants

Entry ID: `fd64effc3432` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 14:37:28 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Noncrossing Partitions) **Field B** (complexity object): Complexity Theory: Read-Twice Branching Program Complexity

Statement:

> ['For every read-twice branching program P, the minimal rank of the associated noncrossing partition graph is O(log size(P)).', 'However, for the trivial read-twice BP that always outputs 0, this rank is Θ(n).', 'Moreover, there exists a constructive mapping from a read-twice BP to its corresponding noncrossing partition graph such that the mapping can be computed in polynomial time.']

Rationale (proposer's reasoning):

> ['Noncrossing partitions are well-studied in algebraic geometry and have connections to matroid theory.', 'They could potentially provide structural information about the complexity of read-twice BPs that is not captured by other known measures.', 'If such a mapping exists, it would offer a novel way to study branching program complexity.']

Taxonomy category: `BP_READTWICE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `c0f093e60cb816b2`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if for at least 80% of the generated read-twice BPs, the ratio between the rank of the associated noncrossing partition graph and the log of the BP size is less than or equal to 1. For falsification, any seed must produce a noncrossing partition graph with a rank exceeding the log of the BP size by more than 3 standard deviations from the mean.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'noncrossing partitions' AND 'read-twice BP size' AND algebraic geometry' - 'minimal rank' AND 'noncrossing partition graph' AND read-twice branching program - algebraic geometry invariants AND complexity theory AND mapping to noncrossing partitions

Top relevant hits considered: - [http://arxiv.org/abs/1604.06009v2] Oriented Flip Graphs and Noncrossing Tree Partitions - [http://arxiv.org/abs/2212.13799v6] Noncrossing partitions of a marked surface - [http://arxiv.org/abs/1904.05573v3] k-Indivisible Noncrossing Partitions - [http://arxiv.org/abs/1509.06942v2] Symmetric Decompositions and the Strong Sperner Property for Noncrossing Partition Lattices - [http://arxiv.org/abs/1409.1534v1] Algorithms in Real Algebraic Geometry: A Survey - [http://arxiv.org/abs/1912.00347v3] Equiresidual algebraic geometry I: The affine theory - [http://arxiv.org/abs/2204.10334v1] Machine Learning Algebraic Geometry for Physics

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_read_twice_bp(n):
        # Generate a random read-twice branching program of size n
        bp = []
        for _ in range(n):
            bp.append(random.choice([0, 1]))
        return bp

    def noncrossing_partition(bp):
        # Construct the noncrossing partition graph from the BP
        if len(bp) == 1:
            return [[0]]

        partitions = []
```

```
    for i in range(len(bp)):
        if bp[i] == 0:
            partitions.append([i])
        else:
            new_partitions = []
            for p in partitions:
                if p[-1] < i:
                    new_partitions.append(p + [i])
                else:
                    new_partitions.append(p)
            partitions = new_partitions
    return partitions

def rank(partition):
    # Compute the rank of the noncrossing partition graph
    n = len(partition)
    M = [[0] * n for _ in range(n)]
    for i, p in enumerate(partition):
        for j in p:
            M[i][j] = 1

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        rank = 0
        for col in range(n):
            pivot_row = -1
            for row in range(rank, m):
                if A[row][col] != 0:
                    pivot_row = row
                    break
            if pivot_row == -1:
                continue

            A[pivot_row], A[rank] = A[rank], A[pivot_row]
            rank += 1

            for row in range(m):
                if row != rank - 1:
                    factor = Fraction(A[row][col], A[rank - 1][col])
                    for j in range(n):
                        A[row][j] -= factor * A[rank - 1][j]

        return rank

    return gaussian_elimination(M)

n = random.randint(5, 40)
bp = generate_read_twice_bp(n)
partition = noncrossing_partition(bp)
rank_value = rank(partition)

metric_name = "Rank/LogSize"
metric_value = rank_value / math.log(n)
instances_tested = 1
conjecture_holds = False
counterexample = ""
```

```
    if n == 1:
        if rank_value != 1:
            counterexample = f"Trivial BP with size {n} has rank {rank_value}, expected 1"
    elif rank_value <= math.log(n):
        conjecture_holds = True

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30))  # Default to f

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_rank) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        result = f"RESULT: SUPPORTED mean={mean_rank} std={std_dev} support_fraction={support_frac
    elif any(r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        result = f"RESULT: SUPPORTED mean={mean_rank} std={std_dev} support_fraction={support_frac
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        counterexample = next((r["counterexample"] for r in results if r["conjecture_holds"]), "")
        result = f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first

    print(result)
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d2905211.py", line 111, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d2905211.py", line 83, in run_trial
    rank_value = rank(partition)
                 ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d2905211.py", line 53, in rank
```

```
    M[i][j] = 1
    ~~~~^^^
IndexError: list assignment index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test with a different seed or fix the error causing the crash in the test code.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11834
2	propose	ollama_remote	glm4:latest	0	0	9950
3	preregistration	ollama_remote	glm4:latest	0	0	5794
4	novelty	ollama_remote	glm4:latest	0	0	4771
5	novelty	ollama_remote	glm4:latest	0	0	6402
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17889
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10734
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10102
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11578
10	judge	ollama_remote	glm4:latest	0	0	13334

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 102389 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/fd64effc3432.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/fd64effc3432.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/fd64effc3432.tar.gz` (if generated)